

CHAPTER 2

(INTRO TO FLUTTER)

Prepared by Nurul Amera

INSTALLING FLUTTER

- Flutter is easy to install.
- Visit: <https://flutter.dev/docs/get-started/install>
- Download:
 - Flutter SDK
 - Dart SDK (included)
 - Dart command-line tool (included)

No need to download Dart separately.

INSTALLATION GUIDE

The Flutter documentation is:

- Very detailed
- Explains all steps clearly
- Includes possible issues and how to fix them

Important: Follow all steps completely to prepare your system for Flutter development.

UPDATING YOUR PATH

1. Add Flutter's bin folder to your system PATH:
 - This lets you run flutter commands from the command line.
 - This step is essential to interact with Flutter and Dart through the terminal.
2. The Flutter website provides a guide to update your PATH variable

CHECKING SETUP – flutter doctor

After installation and PATH setup, run this commane (command prompt)

C:\src\flutter>flutter doctor

This will:

Check Flutter SDK

- Detect Android/iOS toolchains
- Detect installed IDEs (VS Code, Android Studio, etc.)
- Check connected devices

If You See Errors...

Don't panic!

Tips to troubleshoot:

- Double-check your PATH
- Make sure it includes flutter/bin
- Restart your terminal after updating PATH

CREATING A NEW PROJECT

 To start a new project, use:

1. `flutter create <output_directory>`
2. `<output_directory>` = your project name
3. A complete sample app will be generated

Example: `flutter create`

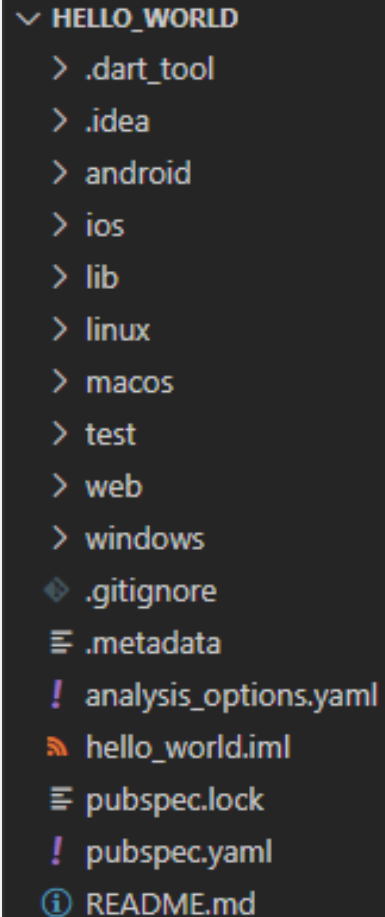
-  Dart package names:
- Use only lowercase letters and underscores
 - Can include numbers, but not at the startMust not use reserved words

Examples:

✓ `hello_flutter`, `my_app1`

✗ `1stApp`, `new`

PROJECT FOLDER STRUCTURE



A screenshot of a file explorer showing the project folder structure. The root folder is 'HELLO_WORLD'. It contains several subfolders: '.dart_tool', '.idea', 'android', 'ios', 'lib', 'linux', 'macos', 'test', 'web', 'windows', and several files: '.gitignore', '.metadata', 'analysis_options.yaml', 'hello_world.iml', 'pubspec.lock', 'pubspec.yaml', and 'README.md'. The folders are expanded to show their contents.

- ✓ HELLO_WORLD
 - > .dart_tool
 - > .idea
 - > android
 - > ios
 - > lib
 - > linux
 - > macos
 - > test
 - > web
 - > windows
 - ◆ .gitignore
 - ≡ .metadata
 - ! analysis_options.yaml
 - 🔥 hello_world.iml
 - ≡ pubspec.lock
 - ! pubspec.yaml
 - 📘 README.md

- android/ and ios/:** Contain platform-specific code. If you're familiar with Android Studio or Xcode, this structure should look familiar.
- hello_flutter.iml:** This is an IntelliJ project file that includes module information (JAVA_MODULE) used by the IDE.
- lib/:** The core directory of your Flutter app. It contains the main Dart file (main.dart) where your app logic begins. We will explore this file in more detail later.
- pubspec.yaml and pubspec.lock:** These files manage the app's configuration.
 - pubspec.yaml defines the app name, version, dependencies (e.g., plugins, fonts, images), and more.
 - pubspec.lock ensures consistency across package versions.
- README.md:** A markdown file used to document your project. It provides setup instructions and important notes so that other developers can easily understand and use your code.
- test/:** Contains all test-related files. You can write unit tests and widget tests here to help ensure your app runs as expected and to catch bugs early during development.

WIDGETS

In Flutter, everything is a widget.

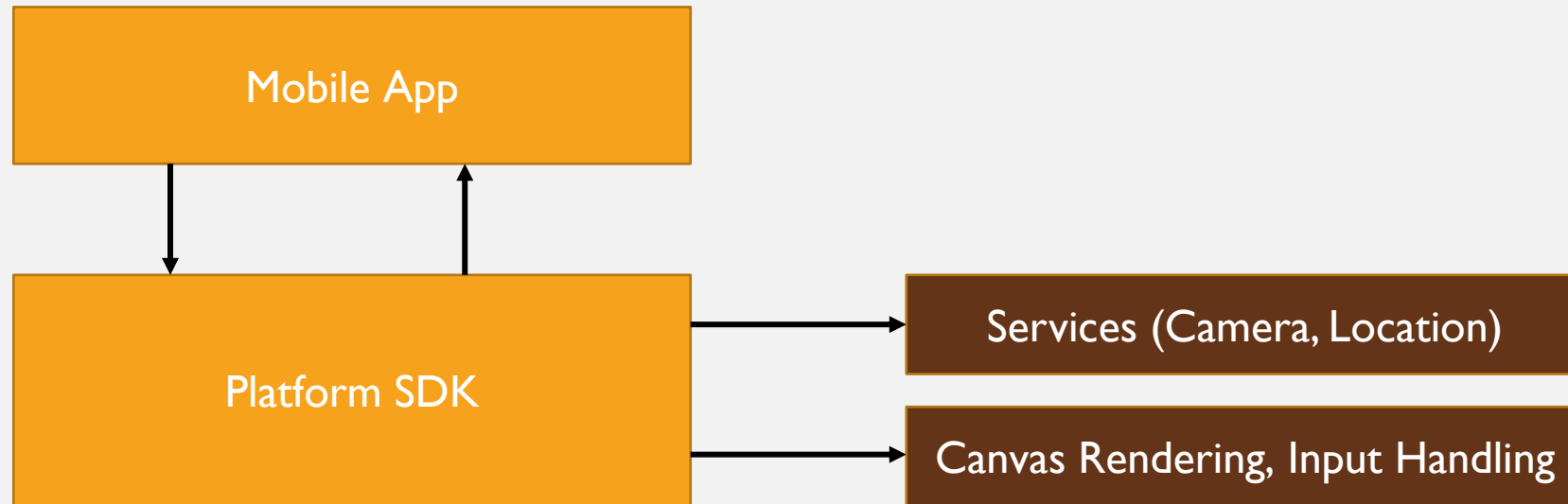
- Widgets represent UI elements and behaviors.

Widgets provide:

- Modularity
- Scalability
- Expressiveness
- Composability & Immutability

FLUTTER RENDERING

1. Flutter handles visual rendering in a unique way.
2. It communicates directly with the platform's SDK (camera, location, etc.).



RENDERING BY FLUTTER ITSELF

1. Flutter takes full control of rendering (no reliance on native UI elements).
2. Only uses the platform SDK for:
 - Services APIs (e.g., Bluetooth, GPS)
 - Canvas to draw UI
 - User input handling

It does:

- Widgets rendering
- Events & gestures
- Communicates via “Bridge Channels” to use platform features

COMPOSABILITY

Composition over inheritance

- Flutter uses composition to build UI — combining smaller widgets.
- Easier to manage and extend than inheritance.

Example:

- A Car is made of an Engine, Wheels, Seats = Composition
- Instead of inheriting everything from a Vehicle = Inheritance

Result:

- ✓ More flexibility
- ✓ Runtime modifications
- ✓ Dependency injection-friendly

EVERYTHING IS A WIDGET

In Flutter, nearly everything is a widget – not just visible elements.

Types of widgets:

- ❑ Visual/Structural: Button, Text
- ❑ Layout-specific: Padding, Row, Column
- ❑ Styling: Theme
- ❑ Interaction: GestureDetector, InkWell

Even the app itself is a widget!

FIRST WIDGET CODE EXAMPLE

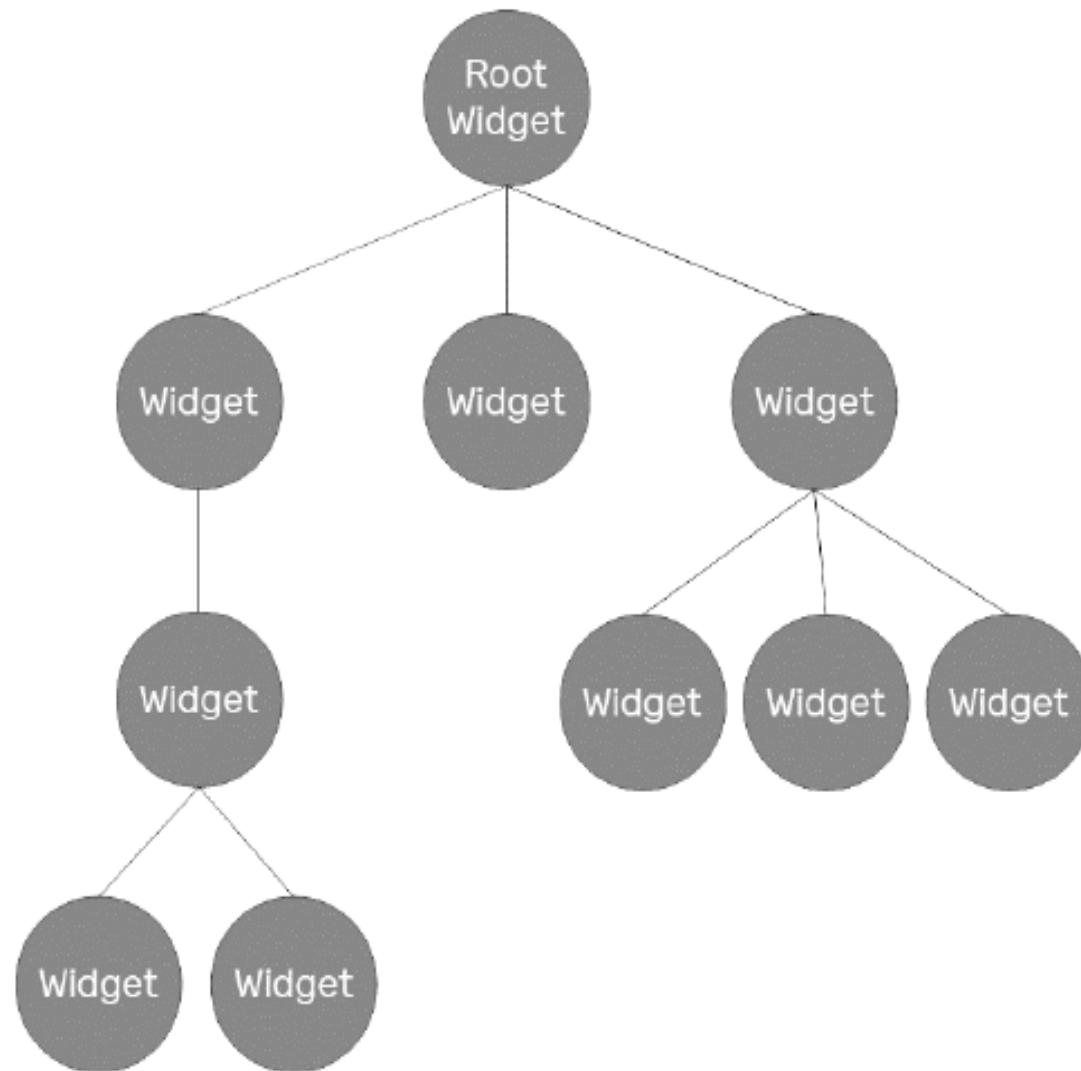
Here's a basic example of a Flutter widget from lib/main.dart:

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Flutter Demo',  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),  
        useMaterial3: true,  
      ), // ThemeData  
      home: const MyHomePage(title: 'Flutter Demo Home Page'),  
    ); // MaterialApp  
  }  
}
```

This example introduces:

- StatelessWidget: doesn't hold
- stateMaterialApp: root of the app
- build() method: returns UI

THE WIDGET TREE



THE WIDGET TREE

1. The Widget Tree is a core part of Flutter's layout system:
 - It is a logical structure of all the widgets in your UI.
 - It is dynamically built and changes based on the widget's child structure.
2. The Widget Tree is built and used during:
 - Layout – determining measurements and positions
 - Rendering – drawing widgets on screen
 - Hit testing – detecting user touch interactions
3. Flutter optimizes rendering by updating only the parts of the tree that changed.

WIDGET TREE NODES

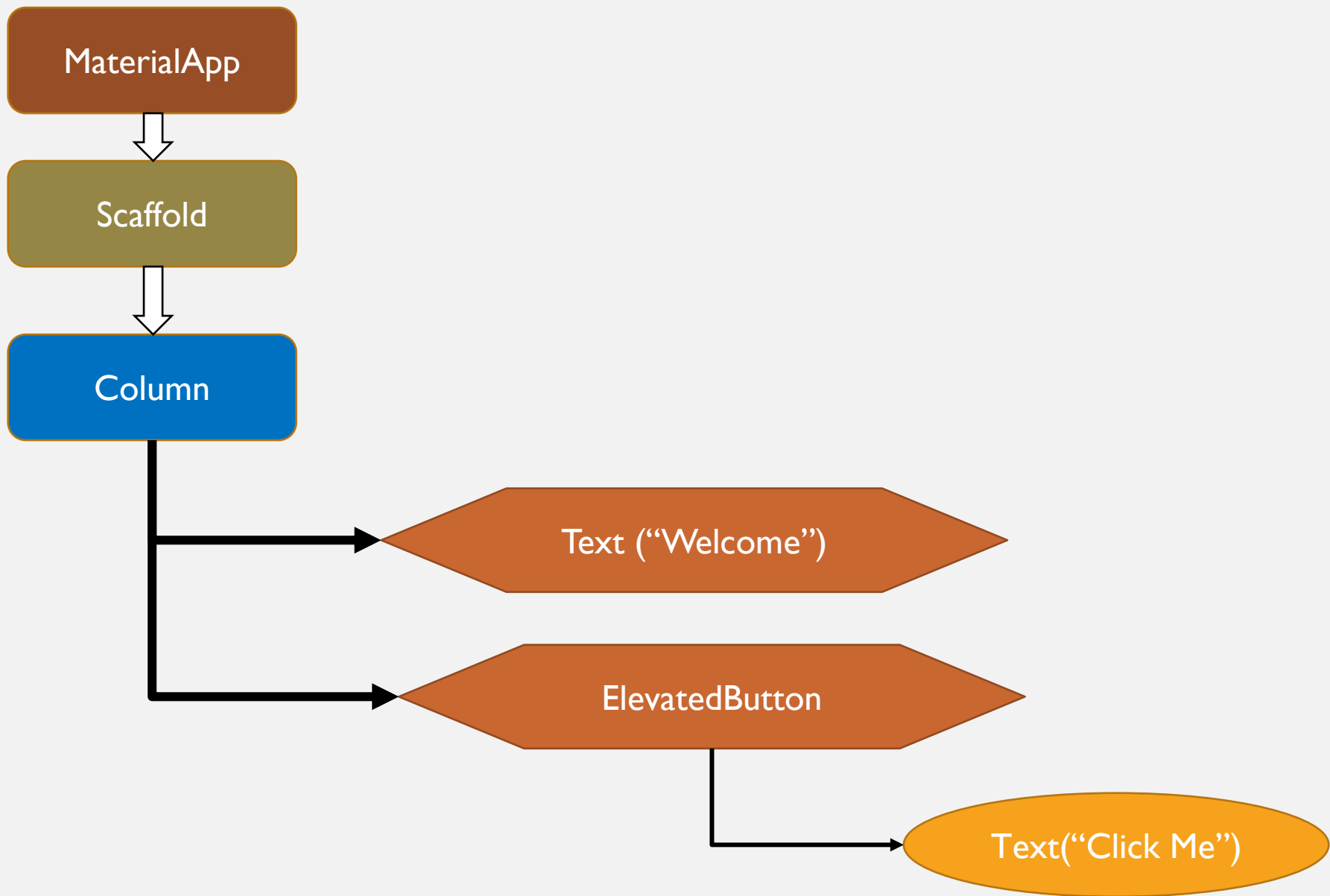
1. Each widget in the tree is a node.

- ❖ Nodes can have children, which form branches of the tree.
- ❖ Every widget has the potential to be recreated when its state changes.

2. Flutter's widget system:

- ❖ Is compositional, not fixed
- ❖ Exposes child or children properties for nesting

3. This dynamic structure lets Flutter be highly flexible and UI-rich.



WIDGET TREE + ELEMENT TREE

The Widget Tree doesn't work alone.

It works together with the Element Tree, which:

- Maps widgets to their corresponding elements
- Helps update or rebuild widgets when state/data changes
- Maintains the widget's position and type on screen

<https://docs.flutter.dev/ui/widgets>

pubspec.yaml



The pubspec.yaml file:

- Defines a Flutter project's metadata, dependencies, and configuration
- Is required in every Flutter project

```
name: hello_world
description: "A new Flutter project."

publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: ^3.5.3
```

- ❑ **name:** Defined when using flutter create
- ❑ **description:** Can be updated with --description flag
- ❑ **version:** Follows Dart package versioning (major.minor.patch+build)

pubspec.yaml

```
environment:  
  sdk: ^3.5.3  
  
dependencies:  
  flutter:  
    sdk: flutter  
  
  cupertino_icons: ^1.0.8
```

- ❑ **environment:** Specifies Dart SDK range supported
- ❑ **dependencies:**
 - Main: Flutter SDK
 - Extras: Like cupertino_icons (icon sets for iOS-style UI)

pubspec.yaml

```
dev_dependencies:  
  flutter_test:  
    sdk: flutter
```

- ❑ **dev_dependencies:** Used only during development/testing
- ❑ **flutter_test:** Comes with Flutter SDK, adds testing utilities

pubspec.yaml

```
flutter:

  uses-material-design: true

  # To add assets to your application, add an assets section, like this:
  # assets:
  #   - images/a_dot_burr.jpeg
  #   - images/a_dot_ham.jpeg

  # An image asset can refer to one or more resolution-specific "variants", see
  # https://flutter.dev/to/resolution-aware-images

  # For details regarding adding assets from package dependencies, see
  # https://flutter.dev/to/asset-from-package

  # To add custom fonts to your application, add a fonts section here,
  # in this "flutter" section. Each entry in this list should have a
  # "family" key with the font family name, and a "fonts" key with a
  # list giving the asset and other descriptors for the font. For
  # example:
  # fonts:
  #   - family: Schyler
  #     fonts:
  #       - asset: fonts/Schyler-Regular.ttf
  #       - asset: fonts/Schyler-Italic.ttf
  #         style: italic
  #   - family: Trajan Pro
  #     fonts:
  #       - asset: fonts/TrajanPro.ttf
  #       - asset: fonts/TrajanPro_Bold.ttf
  #         weight: 700
  #
```

This section allows configuration of resources like:

- ☐ Material icons
- ☐ Assets (images, sound, etc.)
- ☐ Fonts

pubspec.yaml

Example: Adding a single image

```
assets:  
  images/home_background.jpeg
```

Example: Including a full folder

 Ending with / includes all files in that folder, but not subfolders.

```
assets:  
  images/
```

To include subfolders:

```
assets:  
  images/  
  images/icons/
```

<https://docs.flutter.dev/ui/assets/assets-and-images>

CHAPTER 2

(WIDGETS – BUILDING LAYOUTS IN FLUTTER)

Prepared by Nurul Amera

STATEFUL VS STATELESS WIDGETS

1. In Flutter, there are two main widget types:

- Stateless Widgets: Immutable and do not hold state
- Stateful Widgets: Hold state and can rebuild when state changes

Why?

UI elements are not static, they often change over time (like a counter or user interaction). Flutter separates widgets based on whether they manage such changes.

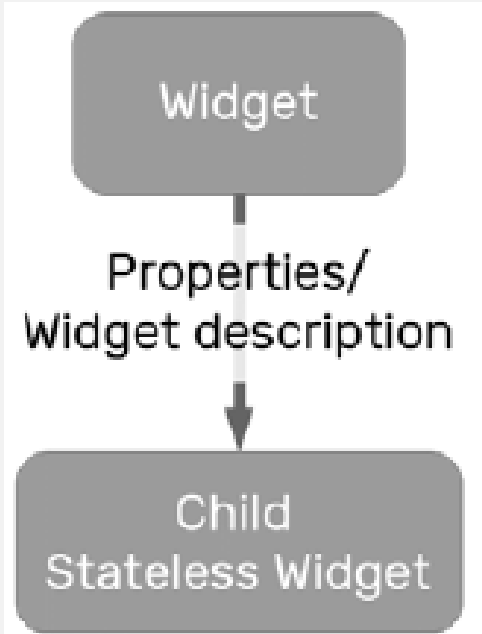
STATELESS VS STATEFUL – WHEN TO USE WHICH?

1. **Stateless:** No internal state, fixed output
2. **Stateful:** Holds mutable state and adapts over time

Developer tips:

- ☐ Use stateless by default unless the widget needs to change dynamically
- ☐ Overusing stateful widgets can lead to performance and maintainability issues

STATELESS WIDGETS



1. Parent widget builds a child StatelessWidget
 2. Passes properties or descriptions
 3. The child cannot change itself, it only receives and uses those inputs
- 🧩 The child widget is fully dependent on the parent's input at creation.

STATELESS WIDGETS

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      title: 'Flutter Demo',  
      theme: ThemeData(  
        colorScheme: ColorScheme.fromSeed(seedColor:  Colors.deepPurple),  
        useMaterial3: true,  
      ), // ThemeData  
      home: const MyHomePage(title: 'Flutter Demo Home Page'),  
    ); // MaterialApp  
  }  
}
```

STATELESS WIDGETS

1. MyApp extends StatelessWidget
2. build() returns MaterialApp
3. No state is held — UI is built from fixed values

What is MaterialApp?

- MaterialApp is a convenience widget:
“A widget that wraps a number of widgets that are commonly required for Material Design applications.”



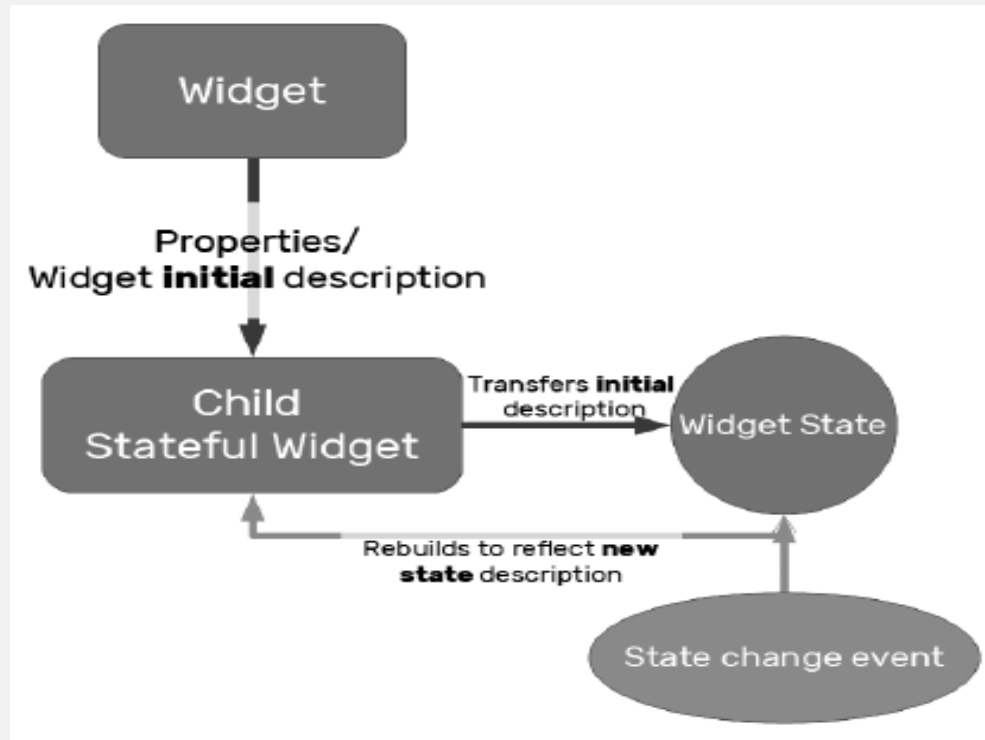
Used to:

- Set app title
- Apply global theme
- Define navigation structure
- Set home widget

Role of BuildContext:

- Gives access to the widget tree structure
- Allows child widgets to inherit theme, styles, etc.
- Helps Flutter know where this widget sits in the tree

STATEFUL WIDGETS



📦 Unlike stateless widgets, stateful widgets can change during their lifetime.

- ❑ They still receive immutable properties from parents.
- ❑ But they manage their own state using a companion State class.
- ❑ When the state changes → Flutter rebuilds the widget tree (UI refresh).

ANATOMY OF A STATEFUL WIDGET

```
class MyHomePage extends StatefulWidget {  
  const MyHomePage({super.key, required this.title});  
  
  final String title;  
  
  @override  
  State<MyHomePage> createState() => _MyHomePageState();  
}
```

- MyHomePage extends StatefulWidget
- Must override createState() to link to its State class
- title is a final property passed from the parent widget

The state class for MyHomePage

```
class _MyHomePageState extends State<MyHomePage> {  
  int _counter = 0;  
  
  void _incrementCounter() {  
    setState(() {  
      _counter++;  
    });  
  }  
}
```

- Holds state variable `_counter`
- `setState()` updates `_counter` and triggers UI rebuild
- `State<MyHomePage>` links this to `MyHomePage` widget

Why Flutter Needs `setState()`:

- A stateful widget must update its UI based on changes over time (e.g., a counter increasing).

How does Flutter know something changed?

- Notifies the framework that something has changed
- Rebuilds the widget with the new state value
- Does not affect unrelated widgets, only redraws what's necessary

Building the UI with State

```
@override
Widget build(BuildContext context) {

  return Scaffold(
    appBar: AppBar(
      backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      title: Text(widget.title),
    ), // AppBar
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          const Text(
            'You have pushed the button this many times:',
          ), // Text
          Text(
            '$_counter',
            style: Theme.of(context).textTheme.headlineMedium,
          ), // Text
        ], // <Widget>[]
      ), // Column
    ), // Center
    floatingActionButton: FloatingActionButton(
      onPressed: _incrementCounter,
      tooltip: 'Increment',
      child: const Icon(Icons.add),
    ), // FloatingActionButton
  ); // Scaffold
}
```

Breakdown of Scaffold Widgets



The build() method for stateful widgets:

- Has the same structure as stateless widgets
- But can access and react to state

In our counter app: We use Scaffold, which holds 3 key widgets:

- **AppBar:**
 - Displays the title
 - Uses the value passed from MyHomePage (widget.title)
- **body:**
 - Centered layout using Center and Column
 - Holds two Text widgets — one fixed and one showing \$_counter
- **floatingActionButton:**
 - Positioned at the bottom right
 - Triggers _incrementCounter() on press

The Full State Update Flow

What happens when you press the "+" button::

- **MyHomePage** builds the layout.
- User taps the button.
- **onPressed** triggers **_incrementCounter()**.
- **_incrementCounter()** calls **setState()**.
- **setState()** updates **_counter**.
- Flutter calls **build()** again with the updated state.



Why This Flow Matters:

- The key to Flutter's power is rebuilding widgets efficiently.
- You don't mutate the UI — you rebuild it.
- **setState()** is the link between logic and UI.
- Once you understand this flow, you understand how all interactive apps in Flutter work.

1. List down the widgets you think you'll need.
 - For example: MaterialApp, Scaffold, AppBar, Text, Column, FloatingActionButton
2. Use this website to help you pick the correct widgets: 👉 <https://docs.flutter.dev/ui/widgets>
 - “This is the official Widget Catalog. Use it to check what widgets are available and what they do.”
3. Draw a Widget Tree.
 - Show which widget goes inside another.
 - Use arrows or lines to connect them like a structure.

